



**UNIVERSIDAD COMPLUTENSE
MADRID**

Actividad de NOSQL

Autor: Gerardo Fructuoso Vidal-Aragón

Profesor/a: Álvaro Bravo

Índice

1. Introducción	2
2. Ejercicio 0	3
3. Ejercicio 1	4
4. Ejercicio 2	5
5. Ejercicio 3	5
6. Ejercicio 4	6
7. Ejercicio 5	6
8. Ejercicio 6	7
9. Ejercicio 7	7
10. Ejercicio 8	7
11. Ejercicio 9	8
12. Ejercicio 10	8
13. Ejercicio 11	9
14. Ejercicio 12	9
15. Ejercicio 13	10
16. Ejercicio 14	11
17. Ejercicio 15	12
18. Ejercicio 16	12
19. Ejercicio 17	13
20. Ejercicio 18	13
21. Ejercicio 19	14
22. Ejercicio 20	14
23. Ejercicio 21	15
24. Ejercicio 22	16
25. Ejercicio 23	17
26. Ejercicio 24	18
27. Ejercicio 25	18
28. Ejercicio 26	18

1. Introducción

En la asignatura de bases de datos NoSQL se ha aprendido a hacer uso de la base de datos de Mongo donde hemos aprendido muchas funcionalidades distintas. La principal diferencia entre una base de datos SQL y NoSQL es que en la base de datos NoSQL no se tienen en cuenta las estructuras de los registros dentro de las colecciones (con lo que es más flexible). Es decir, si una colección tiene muchos registros con 5 campos, se podría incluir otro registro que tenga 6 campos sin problema. Además de ser flexibles, permiten ser más escalables que las SQL.

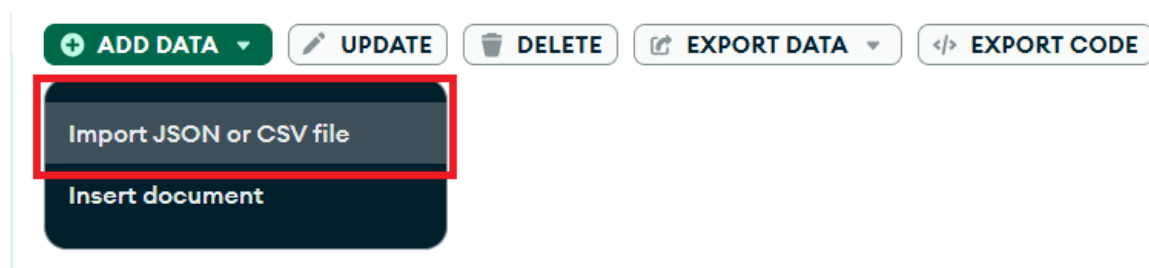
2. Ejercicio 0

En este ejercicio se va a realizar la carga de un archivo JSON dentro de la colección llamada “Movies”. El archivo se encuentra en [este link](#).

Para ello, primero se tiene que crear la colección llamada “Movies” dentro de la base de datos. Accedemos a la aplicación de “MongoDB Compas” y dentro de la base de datos ya creada “Master”, creamos esta nueva colección llamada “Movies”:



Una vez creada esta colección, la seleccionamos para poder importar los datos del JSON dentro de esta colección:



Seleccionamos el JSON que se quiere importar, y se le da a “import”. Una vez importado, aparecen todos los datos del JSON dentro de nuestra colección:

The screenshot shows the MongoDB Compass interface for the 'Movies' collection. The top navigation bar includes 'Local > Master > Movies' and a button to 'Open MongoDB shell'. Below this, tabs for 'Documents', 'Aggregations', 'Schema', 'Indexes', and 'Validation' are visible. The 'Documents' tab is active, showing a list of documents. The first document is expanded, showing its structure: `{ '_id': ObjectId('69ce214237837d5b8e99b3ef'), 'title': 'Caught', 'year': 1900, 'cast': Array (empty), 'genres': Array (empty) }`. The second document is 'After Dark in Central Park' and the third is 'Buffalo Bill's Wild West Parad'.

3. Ejercicio 1

Analizamos colección con un *find* de todo:

```
use Master;
db.Movies.find()
```

Se muestra lo siguiente:

The screenshot shows the MongoDB Compass interface with the 'Movies' collection displayed in a table view. The table has columns for '_id', 'title', 'year', 'cast', and 'genres'. The first document is highlighted in yellow. The table contains 17 documents.

	_id	title	year	cast	genres
1	69ce214237837d5b8e99b3ef	Caught	1900	Array[0]	Array[0]
2	69ce214237837d5b8e99b3f0	After Dark in Central Park	1900	Array[0]	Array[0]
3	69ce214237837d5b8e99b3f1	Buffalo Bill's Wild West Parad	1900	Array[0]	Array[0]
4	69ce214237837d5b8e99b3f2	The Enchanted Drawing	1900	Array[0]	Array[0]
5	69ce214237837d5b8e99b3f3	Clowns Spinning Hats	1900	Array[0]	Array[0]
6	69ce214237837d5b8e99b3f4	Capture of Boer Battery by British	1900	Array[0]	Array[2]
7	69ce214237837d5b8e99b3f5	Feeding Sea Lions	1900	Array[1]	Array[0]
8	69ce214237837d5b8e99b3f6	How to Make a Fat Wife Out of Two Lean C	1900	Array[0]	Array[1]
9	69ce214237837d5b8e99b3f7	Searching Ruins on Broadway, Galveston, f	1900	Array[0]	Array[0]
10	69ce214237837d5b8e99b3f8	The Tribulations of an Amateur Photograp	1900	Array[0]	Array[0]
11	69ce214237837d5b8e99b3f9	Trouble in Hogan's Alley	1900	Array[0]	Array[1]
12	69ce214237837d5b8e99b3fa	Boarding School Girls' Pajama Parade	1900	Array[0]	Array[0]
13	69ce214237837d5b8e99b3fb	Two Old Sparks	1900	Array[0]	Array[1]
14	69ce214237837d5b8e99b3fc	The Wonder, Ching Ling Foo	1900	Array[1]	Array[1]
15	69ce214237837d5b8e99b3fd	Watermelon Contest	1900	Array[0]	Array[1]
16	69ce214237837d5b8e99b3fe	An Affair of Honor	1901	Array[0]	Array[0]
17	69ce214237837d5b8e99b3ff	Another Job for the Undertaker	1901	Array[0]	Array[0]

4. Ejercicio 2

Hacemos un conteo de todos los registros que tiene la colección:

```
db.Movies.find().count()
```

El resultado es el siguiente:

Result x	Find x	Result (1) x
0.006 s		
1	28795	

5. Ejercicio 3

Para este ejercicio se va a insertar una película:

```
db.Movies.insertOne({"title": "test", "year":1800, "cast":[], "genres": []})
```

Resultado:

Result x	Find x	Result (1) x	Result (2) x
0.031 s			
1	{		
2	"acknowledged" : true,		
3	"insertedId" : ObjectId("69ce278b3485d69ae799776f")		
4	}		

6. Ejercicio 4

Se borra la película insertada en el anterior ejercicio:

```
var pelicula = {"title": "test", "year":1800, "cast":[], "genres": []}  
db.Movies.deleteOne(pelicula)
```

Se muestra el resultado:

Result ✕	Find ✕	Result (1) ✕	Result (2) ✕	Result (3) ✕
0.096 s				
1	{			
2		"acknowledged" : true,		
3		"deletedCount" : 1		
4	}			

7. Ejercicio 5

Se obtienen el conteo de todos los actores que se llamen “and”:

```
var query_and = {"cast": {$all: ['and']}}
db.Movies.find(query_and).count()
```

Se muestra el resultado:

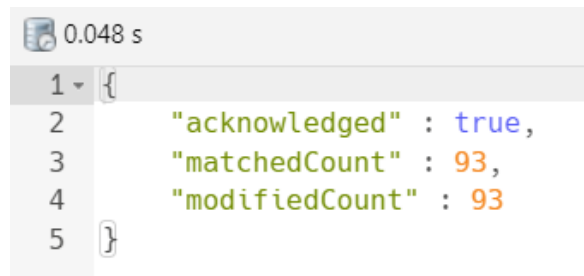
0.046 s	
1	93

8. Ejercicio 6

Se eliminan todos los actores que se llamen ‘and’ ya que están puestos por error:

```
var query_and = {"cast": {$all: ['and']}}
db.Movies.updateMany(query_and,{$pull: {'cast': 'and'}})
```

Se muestra el resultado:



9. Ejercicio 7

Se muestra la cantidad de películas cuyo array de actores está vacío:

```
var query_size0 = {"cast": {$size: 0}}
db.Movies.find(query_size0).count()
```

Se muestra el resultado:

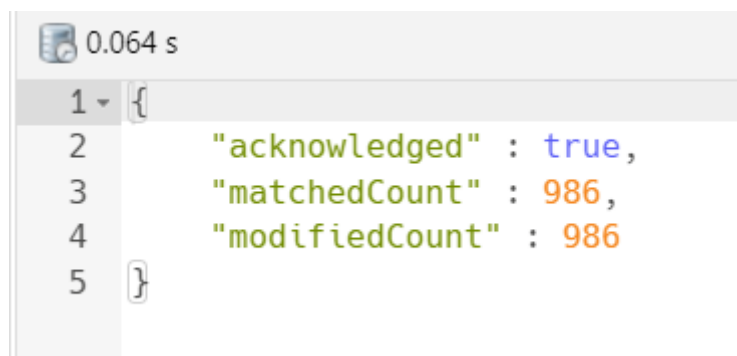


10. Ejercicio 8

Donde el array de actores está vacío, se añade un elemento al array que es 'Undefined':

```
var query_size0 = {"cast": {$size: 0}}
db.Movies.updateMany(query_size0, {$push: {'cast': 'Undefined'}})
```

Resultado:



11. Ejercicio 9

Contar los registros cuyo array de géneros está vacío:

```
var query_size_genres0 = {"genres": {$size: 0}}
db.Movies.find(query_size_genres0).count()
```

Se muestra el resultado:



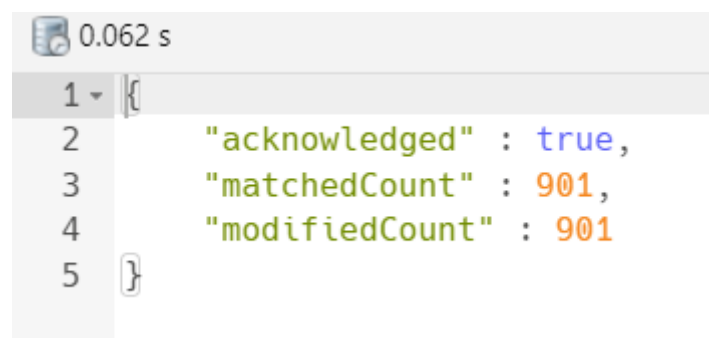
0.042 s
1 901

12. Ejercicio 10

Donde el array de géneros está vacío, se añade un elemento al array que es 'Undefined':

```
var query_size_genres0 = {"genres": {$size: 0}}
db.Movies.updateMany(query_size_genres0, {$push: {'genres': 'Undefined'}})
```

Se muestra resultado:



0.062 s
1 {
2 "acknowledged" : true,
3 "matchedCount" : 901,
4 "modifiedCount" : 901
5 }

13. Ejercicio 11

Se quiere mostrar el año más reciente que tenemos sobre todas las películas:

```
var query_anno_reciente = {}
```



```
var projection_anno = {"year":1, "_id": 0}
db.Movies.find(query_anno_reciente,projection_anno).sort({'year': -1}).limit(1)
```

Se muestra resultado:

Movies	0.045 s	1 Doc
1	{	
2	"year" : 2018	
3	}	

14. Ejercicio 12

Se quiere mostrar la cantidad de películas que han salido en los últimos 20 años (teniendo en cuenta la película del año 2018 vista en el anterior ejercicio):

```
db.Movies.aggregate([
  {$match : {'year': {$gt: (2018-20)}}},
  { $group: {'_id': '$year', 'total' : {$sum: 1}}}
]).sort({'_id': 1})
```

Resultado:

	_id * 	total 
1	1999	221
2	2000	213
3	2001	226
4	2002	232
5	2003	221
6	2004	239
7	2005	228
8	2006	425
9	2007	304
10	2008	206
11	2009	229
12	2010	210
13	2011	201
14	2012	295
15	2013	376
16	2014	214
17	2015	130
18	2016	143
19	2017	238
20	2018	236

15. Ejercicio 13

En este ejercicio se tienen que mostrar la cantidad de películas en la década de los 60:

```
db.Movies.aggregate([
  {$match : {$and: [{'year': {$gte: 1960}}, {'year': {$lte: 1969}}]}},
  { $group: {'_id': '$year', 'total' : {$sum: 1}}}
]).sort({'_id': 1})
```

Resultado:

 Movies	 0.045 s	10 Docs
	_id * ▾	total ▾
1	1960	162
2	1961	150
3	1962	144
4	1963	140
5	1964	151
6	1965	129
7	1966	131
8	1967	127
9	1968	143
10	1969	137

16. Ejercicio 14

Se pide obtener el año o los años que tenga/n la mayor cantidad de películas:

```
var year_most_total = db.Movies.aggregate([
  { $group: { '_id': '$year', 'total' : { $sum: 1 } } }
]).sort({'total': -1}).limit(1).toArray()
//una vez sacamos el total más alto, buscamos por este valor:
var total_maximo = year_most_total[0].total
total_maximo
db.Movies.aggregate([
  { $group: { '_id': '$year', 'total' : { $sum: 1 } } },
  { $match : { 'total': { $eq : total_maximo } } }
]).sort({'total': 1})
```

Se muestra el resultado:

Movies 0.057 s 1 Doc		
	_id * ▴ ▾	total ▴ ▾
1	1919	634

17. Ejercicio 15

Se pide lo mismo que en el ejercicio anterior, pero a la inversa, sacando los que menos películas tienen:

```
var year_least_total = db.Movies.aggregate([
  { $group: { '_id': '$year', 'total' : {$sum: 1}}}
]).sort({'total': 1}).limit(1).toArray()
//una vez sacamos el total más alto, buscamos por este valor:
var total_minimo = year_least_total[0].total
total_minimo
db.Movies.aggregate([
  { $group: { '_id': '$year', 'total' : {$sum: 1}}},
  {$match : {'total': {$eq : total_minimo}}}
]).sort({'total': 1})
```

Se muestra el resultado obtenido:

	_id * ▴ ▾	total ▴ ▾
1	1902	7
2	1907	7
3	1906	7

18. Ejercicio 16

Se pide guardar en una nueva colección llamada 'Actors' realizando un 'unwind':

```
var fase_unwind = {$unwind: '$cast'}
var query_no_repetidos = {'_id': 0} //quitamos el id para no generar errores de duplicados
```

```

var fase_no_repetidos = { $project: query_no_repetidos}
var fase_out = {$out: 'actors'}
var etapas = [fase_unwind,fase_no_repetidos,fase_out]
db.Movies.aggregate(etapas)
//probamos y contamos
db.actors.find().count()

```

Mostramos resultados:

0.004 s

1	83224
---	-------

19. Ejercicio 17

Sobre la colección creada en el ejercicio anterior, tenemos que obtener los 5 actores que han participado en más películas:

```

db.actors.aggregate([
  {$match: {'cast': {$ne: 'Undefined'}}},
  {$group: {'_id': '$cast', 'total_pelis': {$sum:1}}}
]).sort({'total_pelis': -1}).limit(5)

```

Se muestran los resultados:

	_id *	total_pelis
1	Harold Lloyd	190
2	Hoot Gibson	142
3	John Wayne	136
4	Charles Starrett	116
5	Bebe Daniels	103

20. Ejercicio 18

Se pide agrupar por película y año la colección de actors mostrando las 5 en la que más años hayan participado:

```
db.actors.aggregate([
  {$group: {'_id': {'title': '$title','year': '$year'}, 'total_actors':
{$sum:1}}}
]).sort({'total_actors': -1}).limit(5)
```

Se muestra el resultado:

	_id		total_actors
	title	year	
1	The Twilight Saga: Breaking Dawn - Part 2	2012	35
2	Anchorman 2: The Legend Continues	2013	33
3	Cars 2	2011	32
4	Avengers: Infinity War	2018	29
5	Grown Ups 2	2013	28

21. Ejercicio 19

Se requiere obtener los 5 actores cuya carrera haya sido más larga mostrando los años de comienzo, fin y la cantidad:

```
db.actors.aggregate([
  {$match: {'cast': {$ne: 'Undefined'}}},
  {$group: {'_id': '$cast','comienza': {$min: '$year'},'termina':{$max:
'$year'}}},
  {$addFields:{'años':{ $subtract: ['$termina','$comienza']}}}
]).sort({'años': -1}).limit(5)
```

Se muestra el resultado:

	_id *	termina	comienza	años
1	Harrison Ford	2017 (2.0K)	1919 (1.9K)	98
2	Gloria Stuart	2012 (2.0K)	1932 (1.9K)	80
3	Kenny Baker	2012 (2.0K)	1937 (1.9K)	75
4	Lillian Gish	1987 (2.0K)	1912 (1.9K)	75
5	Mickey Rooney	2006 (2.0K)	1932 (1.9K)	74

22. Ejercicio 20

Sobre la colección de 'actors' se requiere crear una nueva de 'genres' usando 'unwind' y mostrando la cantidad de registros:

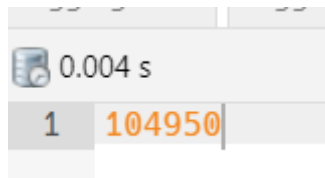
```
var fase_unwind = {$unwind: '$genres'}
```

```

var query_no_repetidos = {'_id': 0} //quitamos el id para no generar errores de
duplicados
var fase_no_repetidos = { $project: query_no_repetidos}
var fase_out = {$out: 'genres'}
var etapas = [fase_unwind,fase_no_repetidos,fase_out]
db.actors.aggregate(etapas)
//probamos y contamos
db.genres.find().count()

```

Se muestran resultados:



0.004 s
1 104950

23. Ejercicio 21

Se requiere mostrar los 5 documentos agrupados por ‘años’ y ‘generos’ que mas números de películas distintas tienen:

```

db.genres.aggregate([
  {$match: {'genres': {$ne: 'Undefined'}}},
  {$group: {'_id': {'generos': '$genres', 'year': '$year'}, 'total_pelis':
{$sum: 1}}}
]).sort({'total_pelis': -1}).limit(5)

```

Se muestra el resultado:

genres 0.096 s 5 Docs			
	_id		total_pelis
	generos	year	
1	Comedy	2012	790
2	Comedy	2013	694
3	Drama	2017	677
4	Drama	2012	657
5	Drama	1919	618

24. Ejercicio 22

Se quieren mostrar los 5 actores que participen en más cantidades de géneros distintos:

```
db.genres.aggregate([
  {$match: {'cast': {$ne: 'Undefined'}}},
  {$group: {'_id': '$cast', 'generos': {$addToSet: '$genres'}}},
  {$addFields: {'numgeneros': {$size: '$generos'}}}
]).sort({'numgeneros': -1}).limit(5)
```

Se muestran las pruebas:

genres 0.166 s 5 Docs			
	_id *	generos	numgeneros
1	Dennis Quaid	Array[20]	20
2	James Mason	Array[19]	19
3	Michael Caine	Array[19]	19
4	Johnny Depp	Array[18]	18
5	Helen Mirren	Array[18]	18

Abrimos un array para verificar:


```

1  [
2    "Disaster",
3    "Romance",
4    "Action",
5    "Thriller",
6    "Suspense",
7    "Adventure",
8    "Dance",
9    "Horror",
10   "Western",
11   "Comedy",
12   "Satire",
13   "Animated",
14   "Fantasy",
15   "Family",
16   "Crime",
17   "Musical",
18   "Science Fiction",
19   "Drama",
20   "Biography",
21   "Sports"
22 ]

```

25. Ejercicio 23

Se quieren mostrar las 5 películas junto con su año en los que más generos diferentes esté clasificada mostrando los géneros y el número:

```

db.genres.aggregate([
  {$match: {'genres': {$ne: 'Undefined'}}},
  {$group: {'_id': {'title': '$title', 'year': '$year'}, 'generos': {$addToSet: '$genres'}}},
  {$addFields: {'numgeneros': {$size: '$generos'}}}
]).sort({'numgeneros': -1}).limit(5)

```

Se muestra el resultado:

genres		0.235 s	5 Docs		
	title	year	generos	numgeneros	
1	American Made	2017	Array[7]	7	
2	Thor: Ragnarok	2017	Array[6]	6	
3	My Little Pony: The Movie	2017	Array[6]	6	
4	Wonder Woman	2017	Array[6]	6	
5	Dunkirk	2017	Array[6]	6	

Se abre un array para verificar:

```

1  [
2    "Historical",
3    "Thriller",
4    "Crime",
5    "Drama",
6    "Action",
7    "Biography",
8    "Comedy"
9  ]

```

26. Ejercicio 24

Query libre: Para este ejercicio se van a contar la cantidad de películas que hay por género (sin tener en cuenta el género 'Undefined') y mostrando solo los 5 primeros:

```

db.genres.aggregate([
  {$match: {'genres': {$ne: 'Undefined'}}},
  {$group: {'_id': '$genres', 'peliculas': {$addToSet: '$title'}}},
  {$addFields: {'total_pelis': {$size: '$peliculas'}}}
]).sort({'total_pelis': -1}).limit(5)

```

Se muestra el resultado:

genres		0.142 s	5 Docs		
	_id *	total_pelis		peliculas	
1	Drama	8352 (8.4K)		Array[8352]	
2	Comedy	7114 (7.1K)		Array[7114]	
3	Western	2874 (2.9K)		Array[2874]	
4	Crime	1464 (1.5K)		Array[1464]	
5	Musical	1137 (1.1K)		Array[1137]	

27. Ejercicio 25

Query libre: Se van a buscar los actores que trabajen en un único género y que ni actores ni generos sean Undefined:

```
db.genres.aggregate([
  {$match: {'cast': {$ne: 'Undefined'}, 'genres': {$ne: 'Undefined'}}},
  {$group: {'_id': '$cast', 'generos': {$addToSet: '$genres'}}},
  {$match: {$expr: {$eq: [{$size: '$generos'}, 1]}}},
  {$addFields: {'numgeneros': {$size: '$generos'}}}
]).sort({'numgeneros': -1})
```

Se muestran los resultados:

	_id *	numgeneros	generos
1	Ron White	1	Array[1]
2	violence	1	Array[1]
3	Trevor Snarr	1	Array[1]
4	Claire Whitney	1	Array[1]
5	Jemima Kirke	1	Array[1]
6	John D. Williams	1	Array[1]

28. Ejercicio 26

Query libre: Vamos a obtener a los actores que hayan participado en más de 10 películas:

```
db.actors.aggregate([
  {$match: {'cast': {$ne: 'Undefined'}}},
  {$group: {'_id': '$cast', 'peliculas': {$addToSet: '$title'}}},
  {$addFields: {'total_pelis': {$size: '$peliculas'}}},
  {$match: {'total_pelis' : {$gt: 10}}}
]).sort({'total_pelis': 1})
```

Se muestra el resultado:

actors		0.366 s	Fetch Count	2000	
	_id	películas			total_pelis
1	William Baldwin	Array[11]			11
2	Patsy Kelly	Array[11]			11
3	Diane Kruger	Array[11]			11
4	Alicia Witt	Array[11]			11
5	Jon Lovitz	Array[11]			11
6	Sebastian Stan	Array[11]			11
7	Willie Nelson	Array[11]			11